Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)
КАФЕДРА «Информационная безопасность» (ИУ8)

Лабораторная работа № 2

«Применение однослойной нейронной сети
с линейной функцией активации для прогнозирования
временных рядов»

по дисциплине «Введение в искусственный интеллект»

Вариант №7

Студент	ИУ8-13М	Петров Илья Евгениевич	
	(Группа)	(И. О. Фамилия)	(Подпись, дата)
Преподаватель		Басараб Михаил Алексеевич	
		(И. О. Фамилия)	(Подпись, дата)

Оценка: _____

Москва, 2026 г.

СОДЕРЖАНИЕ

1 Цель работы 3
2 Постановка задачи 4
3 Теоретические сведения 5
4 Программная реализация 6
5 Исследование влияния числа эпох M 7
6 Исследование влияния нормы обучения η 8
7 Исследование влияния размера окна p 9
ЗАКЛЮЧЕНИЕ 10
ПРИЛОЖЕНИЕ А Графики краткосрочного прогноза 11
ПРИЛОЖЕНИЕ Б Листинг программы 17

1 Цель работы

Изучить применение однослойной нейронной сети с линейной функцией активации для прогнозирования временного ряда и исследовать влияние числа эпох обучения, нормы обучения и размера окна на погрешность краткосрочного прогноза.

2 Постановка задачи

Для варианта 7 используется временной ряд

$$x(t) = 0{,}4\sin(0{,}3t) + 0{,}5 \quad (1)$$

на исходном интервале [−4; 4]. На этом интервале формируется $N = 20$ равномерно расположенных отсчётов. После обучения сети необходимо выполнить краткосрочный прогноз следующих N значений временного ряда. На каждом шаге прогнозируется один следующий отсчёт по p предыдущим реальным значениям ряда.

Вычислительный эксперимент проводится для параметров, приведённых в таблице 1.

Таблица 1 – Параметры вычислительного эксперимента

Параметр	Значение
N	20
a	−4
b	4
Базовый размер окна p	6
Базовая норма обучения η	1,0
Базовое число эпох M	4000
Исследуемые M	500; 1000; 2000; 3000; 4000
Исследуемые η	0,1; 0,3; 0,5; 0,7; 1,0
Исследуемые p	3; 4; 5; 6; 7; 8; 9

3 Теоретические сведения

При линейной функции активации выход нейрона совпадает с его комбинированным входом:

$$y = f(\text{net}) = \text{net}. \tag{2}$$

Для окна длины p прогнозируемое значение формируется как линейная комбинация p предыдущих отсчётов:

$$\widetilde{x_i} = w_0 + \sum_{j=1}^{p} w_j x_{i-p+j-1}. \tag{3}$$

Вес смещения принимается равным нулю: $w_0 = 0$.

Обучение выполняется по правилу Видроу–Хоффа. Для линейной функции активации производная равна единице, поэтому изменение веса определяется выражением

$$\Delta w_j = \eta \delta x_j, \tag{4}$$

где η – норма обучения, $\delta = x_i - y_i$ – ошибка на текущем обучающем примере.

Обучающая выборка формируется из исходных N отсчётов ряда сдвигом окна длины p слева направо; на каждой эпохе окно проходит всю выборку, после чего процесс повторяется M раз. Прогноз выполняется на один шаг вперёд: для каждой следующей точки окно составляется из p последних реальных значений ряда, а полученное предсказание на вход сети не подаётся.

Погрешность прогноза вычисляется как

$$\varepsilon = \sqrt{\sum_{i=1}^{N} (x(t_i) - \tilde{x}(t_i))^2}. \tag{5}$$

4 Программная реализация

Программа реализована на языке Python, графики строятся средствами библиотеки matplotlib. Значения ряда формируются по заданной функции в $N = 20$ равноотстоящих точках интервала $[a; b]$. Для каждого набора параметров веса обнуляются, сеть обучается заново в течение M эпох по правилу Видроу–Хоффа, после чего строится прогноз на N следующих точек и вычисляется погрешность ε.

В программе выполнены три серии экспериментов:

– изменение числа эпох M при фиксированных $p = 6$ и $\eta = 1{,}0$;

– изменение нормы обучения η при фиксированных $p = 6$ и $M = 4000$;

– изменение размера окна p при фиксированных $\eta = 1{,}0$ и $M = 4000$.

Графики краткосрочного прогноза для каждого исследуемого значения M и p приведены в приложении А. Полный листинг программы приведён в приложении Б.

5 Исследование влияния числа эпох *M*

Число эпох M изменялось при фиксированных $p = 6$ и $\eta = 1{,}0$. Полученные значения погрешности приведены в таблице 2.

Таблица 2 – Зависимость погрешности от числа эпох

M	Погрешность ε
500	0,146346
1000	0,140206
2000	0,128448
3000	0,117675
4000	0,107806

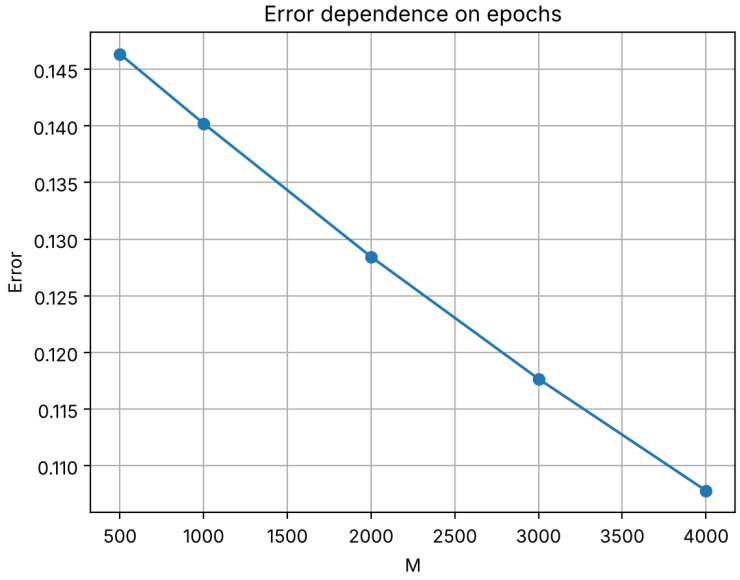


Рисунок 1 – Зависимость погрешности прогноза от числа эпох M

В исследованном диапазоне увеличение числа эпох последовательно уменьшает погрешность: с 0,146346 при $M = 500$ до 0,107806 при $M = 4000$. Все соответствующие прогнозные графики приведены в приложении А.

6 Исследование влияния нормы обучения η

Норма обучения η изменялась при фиксированных $p = 6$ и $M = 4000$. Результаты приведены в таблице 3.

Таблица 3 – Зависимость погрешности от нормы обучения

η	Погрешность ε
0,1	0,190453
0,3	0,146743
0,5	0,130124
0,7	0,120235
1,0	0,107806

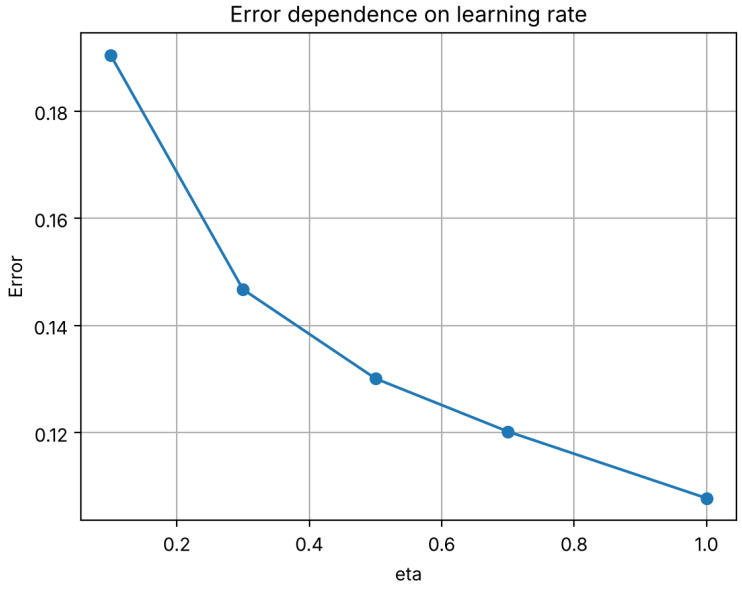


Рисунок 2 – Зависимость погрешности прогноза от нормы обучения η

Увеличение η от 0,1 до 1,0 уменьшает погрешность с 0,190453 до 0,107806. При малой норме обучения за то же число эпох сеть обучается хуже.

7 Исследование влияния размера окна p

Размер окна p изменялся при фиксированных $\eta = 1{,}0$ и $M = 4000$. Полученные результаты приведены в таблице 4.

Таблица 4 – Зависимость погрешности от размера окна

p	Погрешность ε
3	0,060039
4	0,082050
5	0,099054
6	0,107806
7	0,106893
8	0,089263
9	0,026832

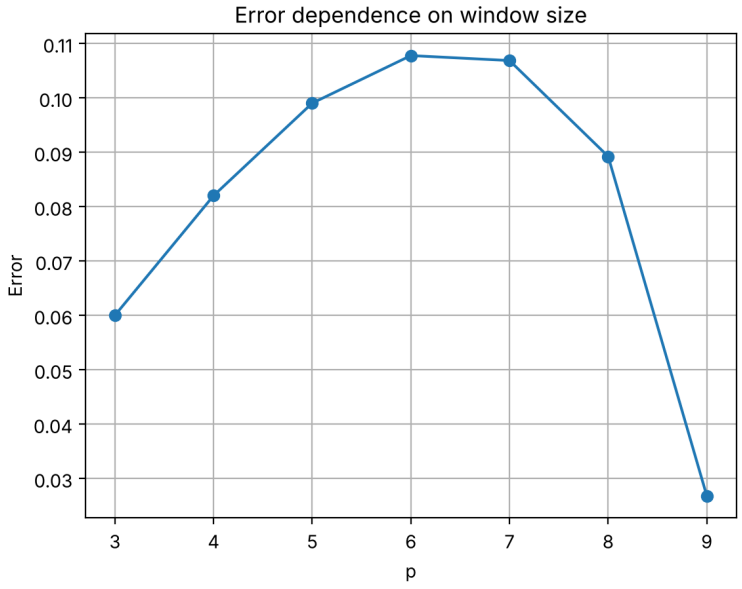


Рисунок 3 – Зависимость погрешности прогноза от размера окна p

Минимальная погрешность получена при $p = 9$: $\varepsilon = 0{,}026832$, наибольшая – при $p = 6$: $\varepsilon = 0{,}107806$. Зависимость немонотонна: при увеличении p от 3 до 6 погрешность растёт, а при дальнейшем увеличении до 9 уменьшается. При этом с ростом p уменьшается число обучающих окон $N - p$: для $p = 9$ их остаётся 11.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы реализована однослойная нейронная сеть с линейной функцией активации для прогнозирования временного ряда варианта 7. Выполнены вычислительные эксперименты по изменению числа эпох обучения, нормы обучения и размера окна.

В выбранном диапазоне увеличение числа эпох M и нормы обучения η сопровождалось уменьшением погрешности прогноза: до 0,107806 при M = 4000 и η = 1,0. Наименьшая погрешность при исследовании размера окна получена для p = 9 (0,026832); зависимость погрешности от p оказалась немонотонной. Для всех исследованных значений M и p построены отдельные графики краткосрочного прогноза на один шаг вперёд (приложение А).

ПРИЛОЖЕНИЕ А

Графики краткосрочного прогноза

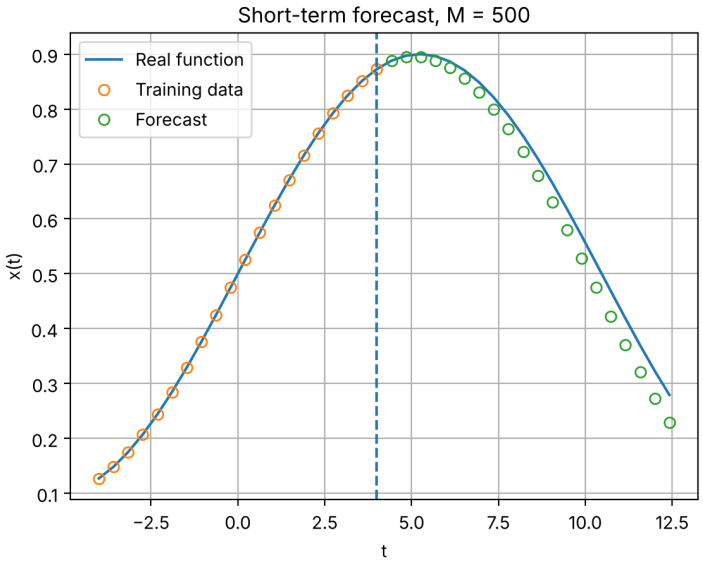


Рисунок А.1 – Прогноз при $M = 500$

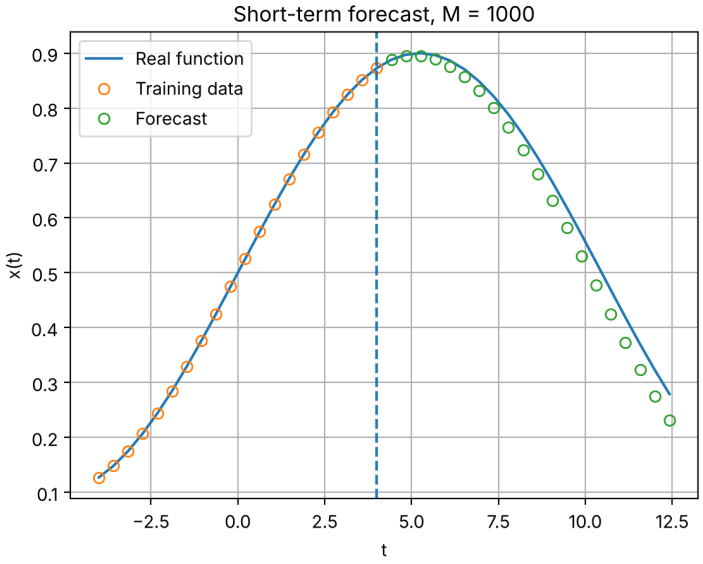


Рисунок А.2 – Прогноз при $M = 1000$

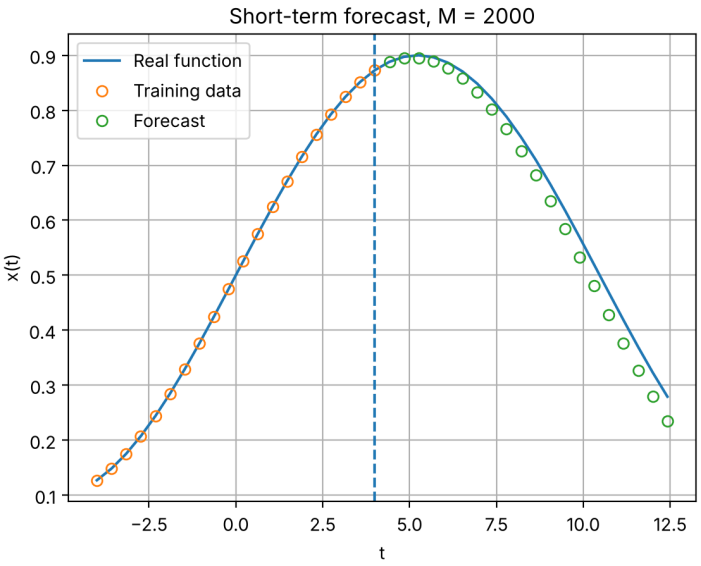


Рисунок А.3 – Прогноз при $M = 2000$

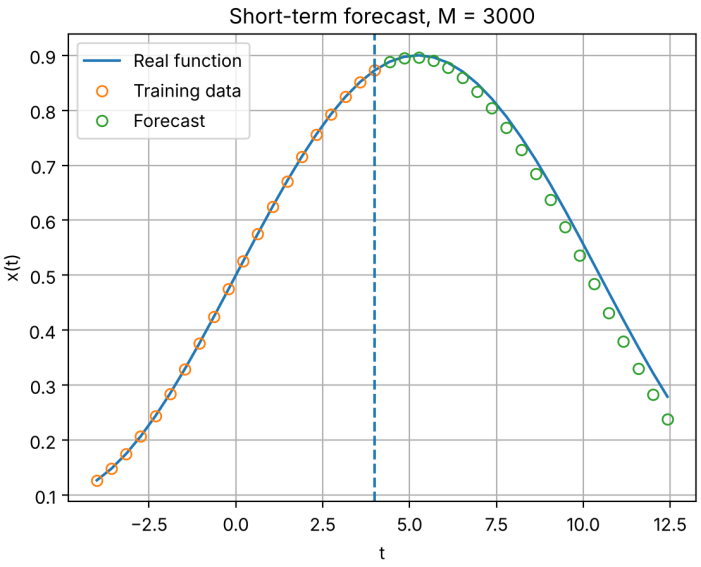


Рисунок А.4 – Прогноз при $M = 3000$

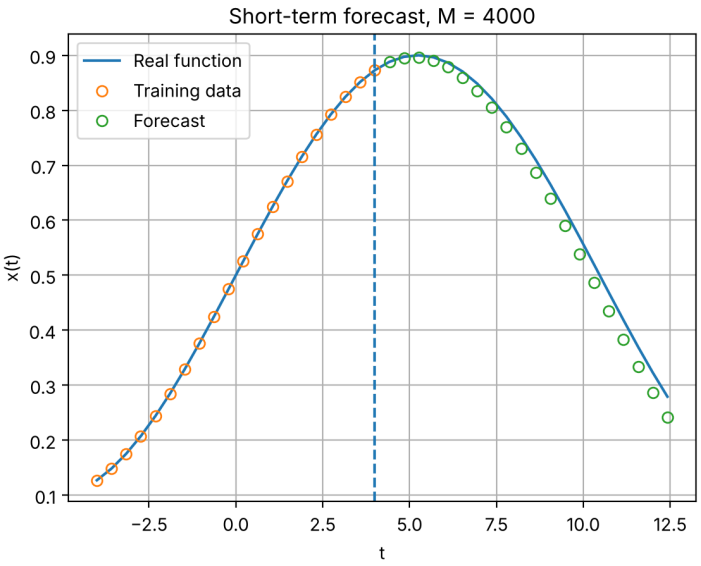


Рисунок А.5 – Прогноз при $M = 4000$

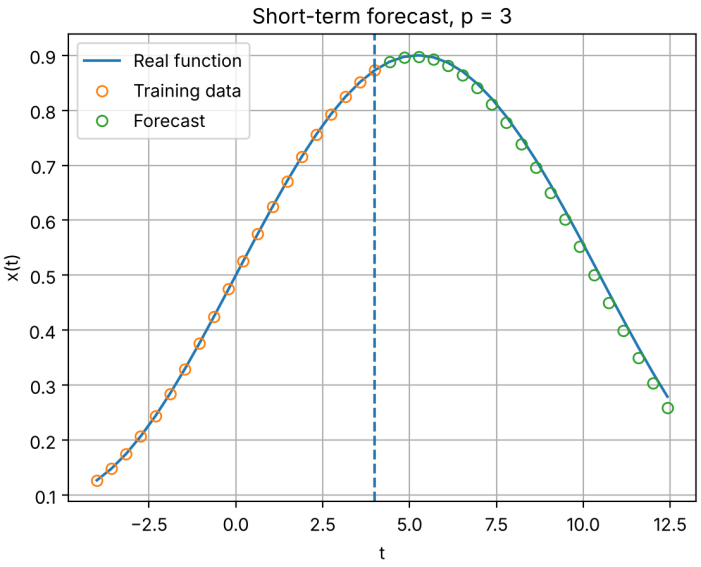


Рисунок А.6 – Прогноз при $p = 3$

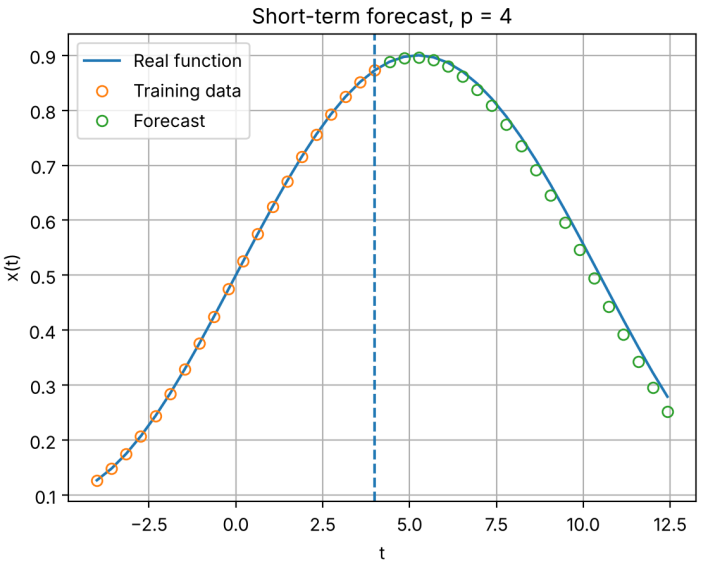


Рисунок А.7 – Прогноз при $p = 4$

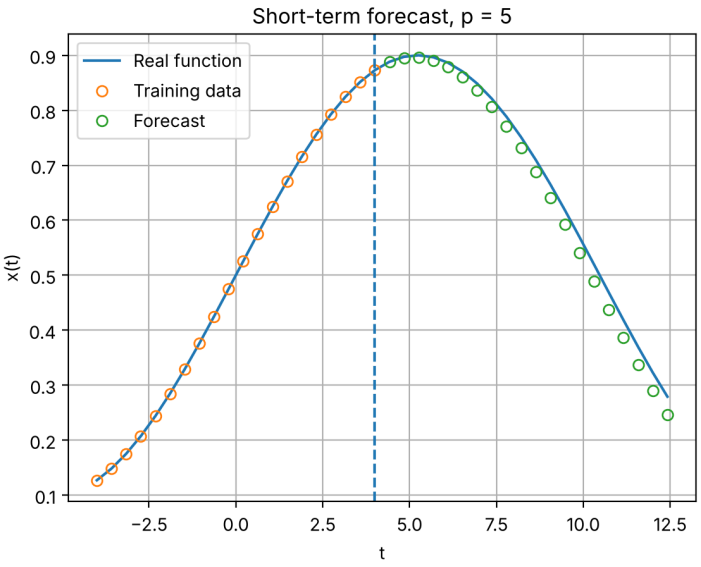


Рисунок А.8 – Прогноз при $p = 5$

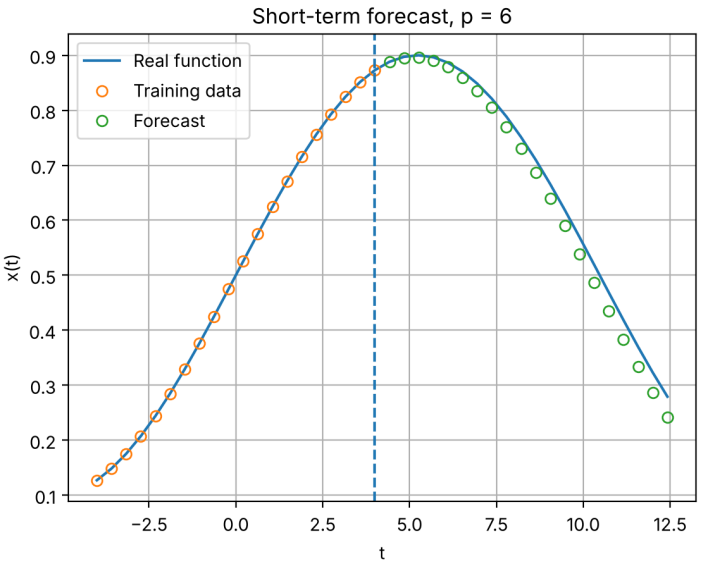


Рисунок А.9 – Прогноз при $p = 6$

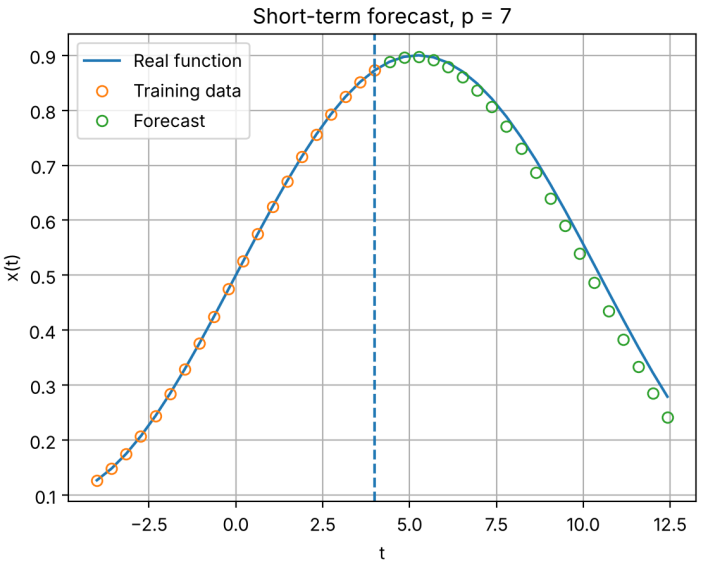


Рисунок А.10 – Прогноз при $p = 7$

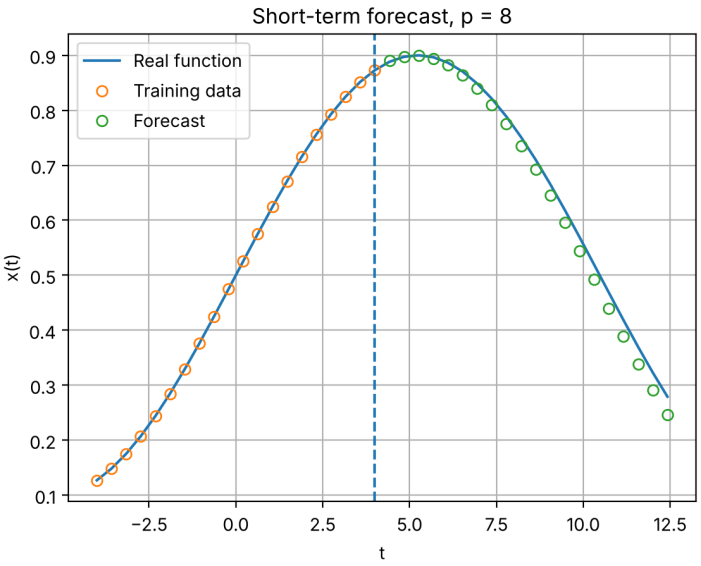


Рисунок А.11 – Прогноз при $p = 8$

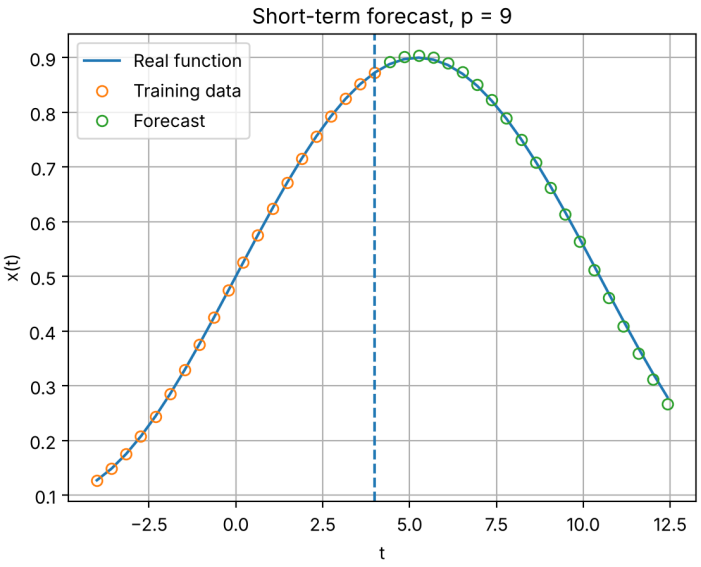


Рисунок А.12 – Прогноз при $p = 9$

ПРИЛОЖЕНИЕ Б

Листинг программы

Листинг Б.1 – lab2.py

```
import math
import os
import matplotlib.pyplot as plt


BASE_P = 6
BASE_ETA = 1.0
BASE_M = 4000


M_VALUES = [500, 1000, 2000, 3000, 4000]
ETA_VALUES = [0.1, 0.3, 0.5, 0.7, 1.0]
P_VALUES = [3, 4, 5, 6, 7, 8, 9]


REAL_INPUTS = True


N = 20
a = -4
b = 4


os.makedirs("graphs", exist_ok=True)



def func(t):
    return 0.4 * math.sin(0.3 * t) + 0.5



def data():
    dt = (b - a) / (N - 1)
    t_lst = []
    x_lst = []
    for i in range(N):
        t = a + i * dt
        x = func(t)
        t_lst.append(t)
        x_lst.append(x)
    return t_lst, x_lst, dt



def train(x_lst, p, eta, M):
    w = [0.0] * p
    for epoch in range(M):
        for i in range(p, N):
            net = 0
            for j in range(p):
                net += w[j] * x_lst[i - p + j]
            delta = x_lst[i] - net
            for j in range(p):
                w[j] += eta * delta * x_lst[i - p + j]
    return w



def forecast(x_lst, w, p, dt):
    history = list(x_lst)
    t_f = []
    x_f = []
```

```
    for k in range(N):
        net = 0
        for j in range(p):
            net += w[j] * history[-p + j]
        t = b + (k + 1) * dt
        t_f.append(t)
        x_f.append(net)
        if REAL_INPUTS:
            history.append(func(t))
        else:
            history.append(net)
    return t_f, x_f



def calc_err(t_f, x_f):
    s = 0
    for i in range(len(x_f)):
        s += (func(t_f[i]) - x_f[i]) ** 2
    return math.sqrt(s)



def run_model(p, eta, M):
    t_lst, x_lst, dt = data()
    w = train(x_lst, p, eta, M)
    t_f, x_f = forecast(x_lst, w, p, dt)
    err = calc_err(t_f, x_f)
    return t_lst, x_lst, t_f, x_f, err



def save_forecast(t_lst, x_lst, t_f, x_f, title, filename):
    real_t = t_lst + t_f
    real_x = []
    for t in real_t:
        real_x.append(func(t))
    plt.figure()
    plt.plot(real_t, real_x, label="Real function")
    plt.plot(t_lst, x_lst, "o", markerfacecolor="none", label="Training data")
    plt.plot(t_f, x_f, "o", markerfacecolor="none", label="Forecast")
    plt.axvline(b, linestyle="--")
    plt.xlabel("t")
    plt.ylabel("x(t)")
    plt.title(title)
    plt.grid()
    plt.legend()
    plt.savefig("graphs/" + filename, dpi=200, bbox_inches="tight")
    plt.close()



def save_error(values, errors, xlabel, title, filename):
    plt.figure()
    plt.plot(values, errors, marker="o")
    plt.xlabel(xlabel)
    plt.ylabel("Error")
    plt.title(title)
    plt.grid()
    plt.savefig("graphs/" + filename, dpi=200, bbox_inches="tight")
    plt.close()



def experiment_M():
    errors = []
    for M in M_VALUES:
        t_lst, x_lst, t_f, x_f, err = run_model(BASE_P, BASE_ETA, M)
        errors.append(err)
        print("M =", M, "err =", err)
```

```
        save_forecast(t_lst, x_lst, t_f, x_f, "Short-
term forecast, M = " + str(M), "forecast_M_" + str(M) + ".png")
    save_error(M_VALUES, errors, "M", "Error dependence on epochs", "error_M.png")
    return errors



def experiment_eta():
    errors = []
    for eta in ETA_VALUES:
        _, _, _, _, err = run_model(BASE_P, eta, BASE_M)
        errors.append(err)
        print("eta =", eta, "err =", err)
    save_error(ETA_VALUES, errors, "eta", "Error dependence on learning rate", "erro
r_eta.png")
    return errors



def experiment_p():
    errors = []
    for p in P_VALUES:
        t_lst, x_lst, t_f, x_f, err = run_model(p, BASE_ETA, BASE_M)
        errors.append(err)
        print("p =", p, "err =", err)
        save_forecast(t_lst, x_lst, t_f, x_f, "Short-
term forecast, p = " + str(p), "forecast_p_" + str(p) + ".png")
    save_error(P_VALUES, errors, "p", "Error dependence on window size", "error_p.pn
g")
    return errors



experiment_M()
experiment_eta()
experiment_p()
```